

Lab 07: Standardise Errors and Request Logs

Maps to: A5, A7

Create predictable error responses and trace a request from status to log event.

AI Vibe Coding Assets

- ai_vibe.py - OpenAI Python SDK runner
- mock-data.json - synthetic request and test context
- mock-database.sqlite - inspectable synthetic SQLite seed database
- Prompts.pdf - first-run and refinement prompts
- working-files/generated/ - isolated AI output for review

First-Run Prompt

Generate request-ID middleware, structured request logging and one public error envelope for 404 and 409 outcomes. Logs may include method, path, status, request id and elapsed time but no API key or body secret. Add tests that correlate the response request id with captured log evidence.

```
python ai_vibe.py --dry-run
export OPENAI_API_KEY="..."
export OPENAI_MODEL="gpt-5-mini"
python ai_vibe.py
pytest -q
```

Detailed procedure

Step 1: Read Prompts.pdf and identify the role, context, constraints, target files and verification checks.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 2: Inspect mock-data.json; remove any personal, confidential or live credential data before prompting.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 3: Run python ai_vibe.py --dry-run to validate and preview the exact SDK request without using API credits.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 4: Set OPENAI_API_KEY in the shell and choose an available OPENAI_MODEL; never paste either value into source files.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 5: Run python ai_vibe.py and inspect the typed CodeBundle written under working-files/generated/.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 6: Review the generated diff and copy only accepted changes into the working application.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 7: Add request-ID middleware that accepts or creates X-Request-ID.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 8: Log method, path, status and elapsed time without logging secrets.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 9: Raise 404 for missing products and 409 for duplicate titles.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 10: Add an exception handler for the public error envelope.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 11: Trigger each error and match the response request_id to the log.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 12: Confirm stack traces are absent from public JSON.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 13: Run pytest -q and the lab acceptance checks; refine the prompt with the observed failure evidence when needed.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Step 14: Save the prompt, model name, generated bundle summary and test result as the lab evidence trail.

Perform the action, observe the output, diagnose any mismatch using the request/status/log evidence, and save one evidence item.

Acceptance criteria

404 and 409 use the same envelope; each response request ID locates one log trace; secrets are absent.